

Alra Architecture and Operations Guide

Architecture, governed retrieval, live operational metrics, security controls and administration

Document control	Value
Document ID	DOORS-AIRA-ARCH-OPS
Version	1.2
Status	Available
Audience	Data Manager, Developer, Security Administrator
System	DOORS Master API and DOORS UI
Classification	Internal operational documentation

Purpose

This guide explains how Alra answers governed DOORS documentation questions and current operational or usage questions. It describes the implemented components, request-routing decisions, local model integration, vector-store lifecycle, metric boundaries, API contracts, security controls, user interface behavior, deployment settings, testing and troubleshooting.

Operating conclusion

Alra uses two evidence paths. Current system facts come from approved read-only database aggregates and are formatted deterministically. Procedures and architecture guidance come from a locally maintained vector knowledge base. The local language model may refine grounded evidence, but it does not generate or execute SQL and is not the source of operational truth.

Contents

- 1 Scope and system boundaries
- 2 Architecture and request flow
- 3 Backend components
- 4 Packages and runtime dependencies
- 5 Vector knowledge base
 - 5.1 Storage architecture and PostgreSQL boundaries
- 6 Operational metrics and insights
- 7 Prompt routing and response types
- 8 Security and privacy controls
- 9 API and UI behavior
- 10 Configuration and deployment
- 11 Reindexing and operations
- 12 Testing and troubleshooting
- 13 Limitations and extension rules

1 Scope and system boundaries

Aira is the DOORS Tutor and Operations Copilot exposed in the management user interface. It supports three request classes: documentation-grounded questions, live operational questions, and mixed questions that need both current metrics and governed procedural context.

Capability	Evidence source	Execution behavior
Documentation answer	Operational Markdown and secured Documentation Hub PDFs	Semantic retrieval followed by an extractive answer or optional local-model refinement
Live status answer	Approved repository methods and parameter-free aggregate SQL owned by the service	Deterministic formatting; no vector retrieval and no model dependency
Mixed answer	Authorized live metrics plus retrieved document chunks	Combined reference context; local model produces the final grounded explanation
Operational insights	Selected live governance, reliability and document-download metrics	Rule-based prioritization with supporting metrics

Aira is read-only. It does not approve queries, change users, execute report SQL, call hosted model services, or interpret retrieved text as instructions.

2 Architecture and request flow

1. The Vue AiraChat component submits a prompt to POST /api/v1/master/aira/chat with the authenticated browser session and optional trace identifier.
2. AiraController obtains the authenticated username and granted authorities and passes them to AiraService.
3. AiraOperationalIntent classifies the request as OPERATIONAL, DOCUMENTATION or MIXED and identifies its domain, metric, status and time period.
4. Operational requests go to AiraOperationalAnswerService and AiraContextService. Documentation requests go to AiraKnowledgeService. Mixed requests use both paths.
5. AiraService returns a structured response and writes an AIRA_CHAT event to unified_audit_logs.
6. The UI renders provenance, capture time, text, and optional table or chart data. Session history remains in browser sessionStorage.

3 Backend components

Component	Responsibility
AiraController	Authenticated chat, refinement, status and knowledge reindex endpoints; maps validation, access and availability errors.
AiraService	Orchestrates routing, evidence assembly, response validation, local-model calls, health status and audit recording.
AiraOperationalIntent	Deterministic phrase and synonym classifier for source, domain, metric, status and period.
AiraOperationalAnswerService	Enforces metric access, selects approved snapshot values, formats live answers and derives operational insights.
AiraContextService	Builds a timestamped read-only snapshot for queries, agents, users, requests, API clients, executions and downloads.
AiraKnowledgeService	Loads or builds embeddings, extracts secured PDFs, chunks sources, searches vectors and persists the local store.
AiraExtractiveFormatter	Builds a fast document-grounded response from retrieved chunks and retains source/page attribution.
AiraAgent	Defines the model system instruction: use supplied reference text only, reject embedded instructions and avoid secrets.
AiraProperties	Binds the doors.aira configuration namespace.

4 Packages and runtime dependencies

Package or platform	Version in project	Use in Alra
Java	21 target	Backend runtime and typed services
Spring Boot	3.2.5	Web, dependency injection, security, JPA and JDBC
LangChain4j	0.31.0	AI service proxy, embeddings and in-memory embedding store
LangChain4j Ollama	0.31.0	Local chat and embedding model adapters
LangChain4j HTTP client JDK	1.11.0	JDK HTTP transport dependency present in the build
Apache PDFBox	3.0.2	Page-aware text extraction from secured PDFs
Jackson	Spring-managed	Vector-store entry counting and JSON handling
Lombok	1.18.40	Boilerplate reduction in backend configuration and services
PostgreSQL driver	Spring-managed	Operational aggregates and audit persistence
Vue	3.5.x	Alra chat interface
Element Plus	2.14.x	Chat controls, badges and result tables
ECharts and vue-echarts	6.1.x and 8.2.x	Metric charts

Version values reflect the current project manifests. Deployment dependency governance remains controlled by the build files and release process.

5 Vector knowledge base

The vector layer is local and sovereign. It uses `LangChain4j InMemoryEmbeddingStore<TextSegment>` while the service is running and serializes the store to a JSON file for restart persistence. It is not a separate database server and does not require a hosted vector service.

Property	Implemented value or default
Embedding model	nomic-embed-text served by local Ollama
Store implementation	LangChain4j InMemoryEmbeddingStore<TextSegment>
Persistent file	security/knowledge/doors-vector-store.json
Markdown source	classpath:knowledge/doors-operational-guide.md
PDF sources	classpath*:secure-docs/*.pdf
PDF chunk limit	1,800 characters with paragraph-aware splitting
Default search limit	4 results
Default minimum score	0.55
Maximum assembled knowledge context	3,500 characters

Index lifecycle

- Startup loads the serialized store when it exists and can be parsed. The entry count is read from its JSON structure for health reporting.
- If the store is missing or invalid, the service attempts a fresh reindex. Indexing may be deferred when the local embedding model is unavailable.
- Reindexing extracts the operational Markdown and every secured PDF. A PDF that produces no searchable text causes reindexing to fail instead of silently publishing incomplete coverage.
- Each PDF segment includes its secure-docs filename, page number and chunk number. These labels support answer citations and auditability.
- A single `AtomicBoolean` prevents concurrent rebuilds. The completed store is serialized to disk and then installed as the active in-memory store.

Retrieval behavior

A query is embedded locally, searched using the configured maximum-result and minimum-score thresholds, and returned as bounded text snippets. Retrieval failures return no snippets and are logged without exposing prompt contents at information level. Operational-only prompts skip this path entirely.

5.1 Storage architecture and PostgreSQL boundaries

Documentation Hub PDF binaries are not stored in PostgreSQL. Their governed source files are application resources under `masterapi/src/main/resources/secure-docs`. The Master API build packages these files inside the executable JAR as `secure-docs/*.pdf` classpath resources. `SecureDocumentationService` resolves the registered filename from that classpath and streams the authorized resource to the caller.

Asset	Physical or logical location	Stored in PostgreSQL?	Lifecycle
Governed PDF binaries	masterapi/src/main/resources/secure-docs; packaged inside the Master API JAR	No	Versioned, reviewed and deployed with the application resource bundle
Document catalogue metadata	SecureDocumentationService.java	No	Maintained in code: ID, title, category, description, topics, roles, version and PDF filename
Active vector index	LangChain4j InMemoryEmbeddingStore<TextSegment> in Master API memory	No	Loaded at startup or rebuilt from governed sources

Asset	Physical or logical location	Stored in PostgreSQL?	Lifecycle
Serialized vector index	security/knowledge/doors-vector-store.json	No	Written to controlled local storage and reloaded after restart
Operational and audit records	PostgreSQL application tables	Yes	Database-managed records such as users, agents, query definitions, executions and audit events

The catalogue is currently code-defined rather than database-driven. Adding a new PDF therefore requires both the governed resource file and its SecureDocumentationService registration. Replacing an existing PDF requires deploying the revised application resource; a database update alone cannot publish it.

Vector embeddings are also outside PostgreSQL. Alra extracts text from the governed Markdown and PDF resources, creates embeddings through the local embedding model, keeps the active index in memory, and persists a serialized copy to doors-vector-store.json. The original PDF is not embedded in that JSON file, and the JSON is not a substitute for the governed source document.

Operational implications

- A PostgreSQL backup or restore does not back up, restore or publish Documentation Hub PDFs or the serialized vector store.
- Protect the application source/build artifact and the controlled vector-store path using the deployment and backup procedures appropriate to each asset.
- After a PDF is added or replaced, redeploy the Master API and reindex Alra so document-grounded answers use the new content.
- Packaging a PDF in the JAR does not make it public. The Documentation Hub endpoint continues to enforce its registered role permissions before streaming the classpath resource.

6 Operational metrics and insights

Domain	Approved metrics
Queries	Total, pending, approved, rejected, active, inactive and active approved
Agents	Total, active, inactive, sandbox and production
Users	Total, active, pending and rejected
Data requests	Total, pending or submitted, approved, rejected, completed and failed
API clients	Total, active and inactive
Executions	Total, successful and failed for today, seven days and thirty days
Document downloads	Total, verified, failed, mismatched, pending receipt and bytes for today, seven days and thirty days
Insights	Pending governance workload, active agents, failed executions, failed downloads and checksum mismatches

Operational answers contain the snapshot capture time. The insight engine prioritizes checksum mismatches, execution or download failures, pending governance work, and absence of active agents. It returns the observation text and the underlying metric records.

7 Prompt routing and response types

Example prompt	Route	Response type
How many queries are pending for approval?	Query metrics, pending count	LIVE_OPERATIONAL
How many documents were downloaded today?	Document-download metrics, today	LIVE_OPERATIONAL
What needs attention today? Give me operational insights.	Operational insight rules	LIVE_OPERATIONAL

Example prompt	Route	Response type
Explain the query approval procedure.	Vector knowledge retrieval	DOCUMENT_GROUNDED
How many queries are pending and who approves them?	Live metrics plus vector knowledge	MIXED

Structured response fields

Field	Meaning
answer	Rendered response text
answerType	LIVE_OPERATIONAL, DOCUMENT_GROUNDED or MIXED
metrics and executionData	Approved metric rows for tables and charts
capturedAt	Timestamp of live evidence collection
knowledgeSnippetsMatched	Number of retrieved vector segments; zero for operational-only answers
grounded	Whether approved evidence was available
refined	Whether optional local-model refinement was requested
executionDisabled	Confirms that model-generated execution is disabled
model	Configured local chat model name

8 Security and privacy controls

- The `/api/v1/master/aira/**` security route is limited to Data Manager and administrative authorities in the current application policy.
- Controller-supplied authorities are forwarded to the operational metric service. Sensitive user, API-client, insight and platform-wide metrics require a manager or security-administration role.
- Operational access uses predefined repository operations and service-owned aggregate SQL. A user prompt is never interpolated into SQL.
- The model receives bounded reference text and an escaped question. Its system instruction treats reference text as data and rejects instructions embedded inside documents.
- The agent is prohibited from exposing vectors, embeddings, passwords, private keys, API secrets and database credentials.
- Chat requests are written as AIRA_CHAT audit records with username, endpoint, status, duration, client IP, trace ID, error code and requested metric identifiers where applicable.
- The UI stores clickable chat checkpoints only in browser sessionStorage, up to thirty items. They are not uploaded as a server-side conversation history and end with the browser session.

9 API and UI behavior

Endpoint	Purpose
POST <code>/api/v1/master/aira/chat</code>	Route and answer a prompt
POST <code>/api/v1/master/aira/chat/refine</code>	Request local-model refinement of retrieved context
GET <code>/api/v1/master/aira/status</code>	Report enablement, model availability, embedding model, vector readiness and segment count
POST <code>/api/v1/master/aira/knowledge/reindex</code>	Rebuild the governed vector knowledge base

The UI shows Live DOORS Status, Operational SOP Grounded, or Live Status plus SOP Grounded provenance. Live responses show an As of timestamp. Structured metric results can be viewed as a table or chart. Live status remains usable when Ollama chat generation is offline, while document and mixed generation depend on configured local model availability.

10 Configuration and deployment

Environment variable	Default	Purpose
DOORS_AIRA_ENABLED	true in current application configuration	Enable local model and knowledge initialization
DOORS_AIRA_BASE_URL	http://127.0.0.1:11434	Local Ollama endpoint
DOORS_AIRA_MODEL	qwen2.5:1.5b	Chat generation model
DOORS_AIRA_TIMEOUT_SECONDS	180	Chat request timeout
DOORS_AIRA_MAX_PROMPT_CHARS	4000	Prompt input limit
DOORS_AIRA_EMBEDDING_MODEL	nomic-embed-text	Embedding model
DOORS_AIRA_VECTOR_STORE_PATH	security/knowledge/doors-vector-store.json	Persistent local vector file
DOORS_AIRA_RAG_MAX_RESULTS	4	Maximum retrieved segments
DOORS_AIRA_RAG_MIN_SCORE	0.55	Minimum similarity score

Deployment checklist

- Install and operate Ollama on the approved local host; pull the configured chat and embedding models before enabling document-grounded service.
- Place the vector-store path on writable, access-controlled local storage. Do not publish or manually edit the serialized embedding file.
- Package secured PDFs under src/main/resources/secure-docs and register them in SecureDocumentationService.
- After adding or replacing a knowledge document, deploy the resource and invoke the authenticated reindex endpoint once Ollama embeddings are available.
- Back up PostgreSQL, deployed documentation artifacts and the vector-store path as separate assets; a database backup does not include the PDFs or embeddings.
- Confirm the status endpoint reports modelAvailable and knowledgeBaseReady as expected, then test one live prompt and one document-grounded prompt.

11 Reindexing and operations

1. Confirm the new or revised PDF is searchable text and has passed document governance review.
2. Deploy the resource and catalogue change to the Master API.
3. Ensure the configured Ollama embedding model is online.
4. Invoke POST /api/v1/master/aira/knowledge/reindex as an authorized Data Manager or administrator.
5. Verify indexedSegments is nonzero and GET /status reports knowledgeBaseReady=true.
6. Ask a specific question covered by the new document and confirm the returned source filename and page.

Do not delete the active vector file during normal operation. If a corrupt store prevents loading, retain it for diagnosis, rebuild through the governed endpoint, and verify coverage before removing the retained copy.

12 Testing and troubleshooting

Symptom	Likely cause	Action
Live question returns a document excerpt	Operational domain or synonym is not present in AiraOperationalIntent	Add the governed domain mapping and an exact regression test; do not broaden generic keywords without tests.

Symptom	Likely cause	Action
Document answer has no relevant content	Vector store is stale, source is absent, or score threshold is too high	Check status, confirm source packaging, reindex, then test retrieval before changing thresholds.
Model is offline but live metrics are needed	Ollama is unavailable	Use operational prompts; they bypass model generation. Restore Ollama for document or mixed answers.
Reindex returns zero or fails	Embedding model unavailable or a source produced no text	Inspect service logs, verify Ollama models and validate PDF text extraction.
Pending metric unexpectedly returns zero	Schema/table query is unavailable or the real state is zero	Compare with the governed dashboard and service logs; do not assume zero proves availability.
Chat history seems incomplete	Entries are scoped to the current browser session	Keep the session open; server-side durable history is intentionally not implemented.

Storage symptom	Cause	Action
PDF is missing after a database restore	PDF binaries are not stored in PostgreSQL	Redeploy the approved Master API artifact containing secure-docs and verify the catalogue entry.
Documentation works but Alra retrieves older text	The serialized vector store predates the deployed PDF	Run the authenticated reindex operation and verify the returned source filename and page.

Required regression coverage

- Exact wording, synonyms, singular/plural and time-window variants for every operational domain.
- Operational prompts return zero knowledge matches and never cite PDFs.
- Documentation prompts do not receive unnecessary live data.
- Mixed prompts include both evidence paths and retain their provenance.
- Role denial returns HTTP 403 and is audited.
- Empty datasets, model unavailability, vector-store corruption and database failures have explicit behavior.

13 Limitations and extension rules

- Intent classification is deterministic phrase matching, not a general natural-language planner. New domains require explicit vocabulary and tests.
- The in-memory vector store is appropriate for the current governed corpus. Larger corpora may require a reviewed persistent vector engine, backup model and migration design.
- Operational metrics are aggregate and read-only. Record-level drill-down requires a separately authorized API and must not be added through free-form SQL generation.
- The local model can still produce an unacceptable response. Aira validates incomplete output and source requirements, retries once where configured, and otherwise fails closed.
- Adding a PDF to the Documentation Hub also makes it eligible for the next vector reindex. Publication and vector activation are separate operational steps.
- Moving PDFs or vector embeddings into PostgreSQL would be an architectural change requiring explicit schema, migration, access-control, backup and performance design; it is not the current implementation.

Review and assurance checklist

- Technical statements match the deployed Alra code and approved configuration.
- Package versions match the project manifests.
- Operational metric definitions and role boundaries have been reviewed.
- The document contains no credentials, private keys, sensitive records or exploitable internal values.
- The PDF renders correctly, is registered in the Documentation Hub and produces searchable text.
- The vector knowledge base is reindexed after deployment and retrieval is verified.
- Related test cases, runbooks and release notes are updated where applicable.

Approval record

Record the approved document version, reviewer, approval reference and effective date in the governed release record. The authenticated Documentation Hub copy supersedes uncontrolled downloaded copies.